

Aperçu du Projet

Dans ce cours, nous allons nous concentrer sur les orchestrateurs axés sur les assets et sur la manière dont ils simplifient la gestion des pipelines de données. Vous allez utiliser **Dagster**, un orchestrateur open-source, pour construire un pipeline de données exemple.

À l'aide des données issues de **NYC OpenData**, vous allez créer un pipeline de données qui :

- **Extraire** les données stockées dans des fichiers Parquet depuis NYC OpenData.
- **Charger** ces données dans une base de données **DuckDB**.
- **Transformer** et préparer les données pour l'analyse.
- **Crée une visualisation** à partir des données transformées.

Si vous êtes bloqué ou souhaitez avancer plus rapidement, consultez le projet finalisé disponible sur **GitHub**.

Prérequis

Pour installer **Dagster**, vous aurez besoin de :

- **Installer Python**. Dagster prend en charge **Python 3.9 à 3.12**.
- Un **gestionnaire de paquets** comme **pip**, **Poetry** ou **uv**. Si vous devez installer un gestionnaire de paquets, consultez les guides d'installation suivants :
 - [pip](https://pip.pypa.io/en/stable/installation/) (<https://pip.pypa.io/en/stable/installation/>)
 - [Poetry](https://python-poetry.org/docs/#installation) (<https://python-poetry.org/docs/#installation>)
 - [uv](https://github.com/astral-sh/uv) (<https://github.com/astral-sh/uv>)

Pour vérifier si **Python** et le **gestionnaire de paquets** sont déjà installés dans votre environnement, exécutez :

```
python --version
pip --version
```

Installation

❑ **Remarque** : Il est fortement recommandé d'installer Dagster dans un **environnement virtuel Python**. Si vous avez besoin d'un rappel sur la création et l'activation d'un environnement virtuel, consultez [cet article de blog](https://realpython.com/python-virtual-environments-a-primer/) (<https://realpython.com/python-virtual-environments-a-primer/>).

Pour installer **Dagster** dans votre environnement Python actuel, exécutez la commande suivante :

```
pip install dagster~=1.9
```

Création du projet Dagster

Créons votre premier projet Dagster ! Pour cela, utilisez la commande `dagster project from-example` afin de cloner le projet officiel **Dagster University** sur votre machine locale.

Pour créer le projet, exécutez :

```
dagster project from-example --example project_dagster_university_start --name dagster_university
```

Après l'exécution de cette commande, un nouveau répertoire nommé **dagster_university** sera créé dans votre répertoire actuel. Ce répertoire contiendra les fichiers constituant votre projet Dagster.

Ensuite, configurez les variables d'environnement par défaut et installez les dépendances Python du projet en exécutant :

```
cd dagster_university
cp .env.example .env
pip install -e ".[dev]"
```

L'option `-e` installe le projet en mode **éditable**, ce qui améliore l'expérience de développement en réduisant le temps nécessaire pour tester une modification. Les principales exceptions sont l'ajout de nouveaux assets ou l'installation de dépendances supplémentaires.

Pour vérifier que l'installation a réussi et que vous pouvez exécuter Dagster en local, lancez :

```
dagster dev
```

Naviguez vers **localhost:3000** (<http://localhost:3000>), où vous devriez voir l'interface utilisateur de Dagster.

- ❑ **Remarque** : La commande `dagster dev` exécutera Dagster en continu jusqu'à ce que vous l'arrêtiez. Pour arrêter le processus en cours d'exécution, utilisez **Control + C** dans le terminal.

Définition de votre premier asset

Dans ce cours, vous utiliserez les données de **NYC OpenData** pour analyser les trajets en taxi à New York. Le premier asset que vous allez définir utilise les données de **TLC Trip Record Data**, qui contient les enregistrements de trajets pour plusieurs types de véhicules. Nous nous concentrerons sur les taxis jaunes.

Votre premier asset, nommé `taxi_trips_file`, va récupérer les données des taxis jaunes pour **mars 2023** et les enregistrer localement.

1. **Ouvrir le fichier `assets/trips.py`** dans votre projet Dagster.
2. **Vérifier que les imports suivants existent déjà** en haut du fichier :

```
import requests
from . import constants
```

3. **Définir une fonction qui récupère les données et les sauvegarde localement :**

```
def taxi_trips_file() -> None:
    """
    Récupère les fichiers Parquet bruts des trajets en taxi.
    Sourced from the NYC Open Data portal.
    """
    month_to_fetch = '2023-03'
    raw_trips = requests.get(
        f"https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_{month_to_fetch}.parquet"
    )

    with open(constants.TAXI_TRIPS_TEMPLATE_FILE_PATH.format(month_to_fetch), "wb") as output_file:
        output_file.write(raw_trips.content)
```

4. **Transformer cette fonction en un asset Dagster :**

- **Importer asset depuis la bibliothèque Dagster :**

```
from dagster import asset
```

- **Ajouter le décorateur `@asset` avant la fonction :**

```
@asset
def taxi_trips_file() -> None:
    """
    Récupère les fichiers Parquet bruts des trajets en taxi.
    """
    month_to_fetch = '2023-03'
    raw_trips = requests.get(
        f"https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_{month_to_fetch}.parquet"
    )

    with open(constants.TAXI_TRIPS_TEMPLATE_FILE_PATH.format(month_to_fetch), "wb") as output_file:
        output_file.write(raw_trips.content)
```

C'est tout ! ☐ Vous venez de créer votre premier **asset Dagster**. En utilisant le décorateur `@asset`, vous pouvez facilement transformer une fonction Python en un asset Dagster.

i Note sur -> None : Il s'agit d'une **annotation de type** en Python indiquant que la fonction ne retourne rien. L'utilisation des annotations de type est fortement recommandée pour rendre le code plus lisible et compréhensible.

Matérialisation de votre asset

Une fois votre asset défini dans le code, l'étape suivante consiste à le **matérialiser**. Lorsqu'un asset est matérialisé, Dagster exécute la fonction associée et enregistre le résultat dans un stockage, comme un entrepôt de données.

Exécution de la matérialisation via l'interface Dagster

Si vous n'avez pas encore l'interface **Dagster UI** en cours d'exécution, démarrez-la en exécutant la commande suivante à la racine de votre projet Dagster :

```
dagster dev
```

Puis, accédez à **localhost:3000** (<http://localhost:3000>) dans votre navigateur.

1. Cliquez sur **"Assets"** dans la barre de navigation supérieure. Vous verrez la liste des assets disponibles dans le projet.
2. Si la page est vide, cliquez sur **"Reload definitions"** pour recharger les définitions des assets.
3. Cliquez sur **"View global asset lineage"** pour visualiser la hiérarchie de votre asset.
4. Cliquez sur le bouton **"Materialize"** pour exécuter la fonction de l'asset et générer le fichier de données.
5. Une boîte de notification violette apparaîtra, indiquant que l'exécution a démarré avec succès.
6. Une fois la matérialisation terminée, accédez au répertoire `data/raw/` dans votre projet et vérifiez la présence du fichier généré :

```
ls data/raw/taxi_trips_2023-03.parquet
```

☐ **Remarque :** Le téléchargement du fichier peut prendre quelques instants.

C'est tout ! ☐ Vous avez maintenant **matérialisé votre premier asset** avec Dagster !

Consultation des détails d'exécution

Maintenant que vous avez matérialisé un asset, il est essentiel de savoir comment consulter les détails d'exécution.

1. Localiser l'exécution de la matérialisation

- o Dans l'interface **Dagster UI**, repérez la section **Materialized** - sur la page de l'asset.
- o Cliquez sur la **date** mise en évidence pour accéder à la page de détails d'exécution.

2. Consulter les logs et informations

- o La page affichera des informations sur l'exécution, y compris les **logs**, les **statuts d'exécution**, et les éventuelles **erreurs**.
- o Vous pouvez utiliser ces détails pour **identifier et résoudre d'éventuels problèmes**.

Si votre exécution avait échoué, cette page vous aiderait à diagnostiquer la cause et à ajuster votre code en conséquence. Parcourez la afin de vous familiariser avec l'UI et un potentiel débogage...

Dépannage des exécutions échouées

Si un asset échoue lors de sa matérialisation, vous pouvez identifier la cause et corriger l'erreur en suivant ces étapes :

1. Déclencher une erreur intentionnelle

- Modifiez `assets/trips.py` en commentant l'importation de `constants`.
- Lancez la matérialisation dans **Dagster UI** et observez l'échec.

2. Analyser les logs d'exécution

- Ouvrez les **Run details** et repérez le statut **Failure**.
- Identifiez l'étape en échec via le message d'erreur dans les logs.

3. Corriger l'erreur

- Réactivez l'importation de `constants` dans `trips.py` et enregistrez.
- Fermez la fenêtre d'erreur dans **Dagster UI**.

4. Relancer l'exécution

- Cliquez sur **Re-execute all** dans **Dagster UI** pour relancer toutes les étapes.
- Vérifiez que la matérialisation aboutit avec succès.

Ces étapes vous permettront de comprendre comment utiliser les logs et les outils de Dagster pour diagnostiquer et corriger les erreurs efficacement.

Création d'un asset `taxi_zones_file`

Pour mettre en pratique ce que vous avez appris, créez un nouvel asset dans `trips.py` qui :

- Est nommé `taxi_zones_file`, qui contiendra un identifiant unique et le nom de chaque zone de taxi à New York.
- Utilise la bibliothèque `requests` pour récupérer les données depuis l'URL suivante :

`https://community-engineering-artifacts.s3.us-west-2.amazonaws.com/dagster-university/data/taxi_zones.csv`
- Stocke les données sous forme de fichier CSV dans `data/raw/taxi_zones.csv`. Le chemin est défini dans `constants.TAXI_ZONES_FILE_PATH`.

Une fois cet asset créé et ajouté à votre projet, vous pourrez le matérialiser de la même manière que l'asset précédent en utilisant l'interface **Dagster UI**.

Création d'un pipeline de données

Jusqu'ici, nous avons vu ce qu'est un asset et comment en créer dans Dagster. Maintenant, nous allons étendre votre projet Dagster pour construire un pipeline de données.

Les pipelines de données sont une suite d'événements produisant des assets. Dagster permet de créer efficacement ces pipelines tout en gérant plusieurs assets interdépendants.

Vous allez maintenant construire un pipeline complet qui :

- Charge les fichiers téléchargés dans une base de données.
- Combine et agrège les données pour générer des métriques sur les trajets en taxi.
- Visualise ces métriques et enregistre un rapport.

Pendant ce processus, vous apprendrez à définir des dépendances entre assets pour structurer vos pipelines efficacement.

Exécution des assets et base de données

Précédemment, nous avons créé deux assets indépendants : `taxi_trips_file` et `taxi_zones_file`. Maintenant, nous allons créer de nouveaux assets qui dépendent de ces fichiers de données.

Avant de travailler avec ces fichiers, il est préférable de les charger dans une base de données pour améliorer l'efficacité et le stockage. Heureusement, le projet est configuré avec **DuckDB**, une base de données intégrée qui facilite l'ingestion et la requête de données.

DuckDB permet notamment d'exécuter des requêtes SQL directement sur des fichiers. Par exemple, pour charger le fichier `taxi_trips_file` dans DuckDB, on pourrait exécuter la requête SQL suivante :

```
CREATE OR REPLACE TABLE trips AS (  
  SELECT  
    VendorID AS vendor_id,  
    PULocationID AS pickup_zone_id,  
    DOLocationID AS dropoff_zone_id,  
    RatecodeID AS rate_code_id,  
    payment_type AS payment_type,  
    tpep_dropoff_datetime AS dropoff_datetime,  
    tpep_pickup_datetime AS pickup_datetime,  
    trip_distance AS trip_distance,  
    passenger_count AS passenger_count,  
    total_amount AS total_amount  
  FROM 'data/raw/taxi_trips_2023-03.parquet'  
);
```

Nous verrons dans la prochaine section comment exécuter cette requête et intégrer ce chargement dans Dagster.

Chargement des données dans la base de données

Maintenant que nous avons défini une requête SQL permettant de charger les données dans **DuckDB**, nous allons utiliser **Dagster** pour gérer la matérialisation de ces assets. En déléguant cette tâche à Dagster, nous pouvons facilement suivre l'évolution des tables et mesurer le temps d'exécution.

Définition de l'asset `taxi_trips`

Ajoutez le code suivant à la fin du fichier `trips.py` :

```

import duckdb
import os

from dagster import asset
from dagster._utils.backoff import backoff

@asset(
    deps=["taxi_trips_file"]
)
def taxi_trips() -> None:
    """
    Le jeu de données brut des trajets en taxi, chargé dans une base de données DuckDB.
    """
    query = """
    CREATE OR REPLACE TABLE trips AS (
        SELECT
            VendorID AS vendor_id,
            PULocationID AS pickup_zone_id,
            DOLocationID AS dropoff_zone_id,
            RatecodeID AS rate_code_id,
            payment_type AS payment_type,
            tpep_dropoff_datetime AS dropoff_datetime,
            tpep_pickup_datetime AS pickup_datetime,
            trip_distance AS trip_distance,
            passenger_count AS passenger_count,
            total_amount AS total_amount
        FROM 'data/raw/taxi_trips_2023-03.parquet'
    );
    """

    conn = backoff(
        fn=duckdb.connect,
        retry_on=(RuntimeError, duckdb.IOException),
        kwargs={
            "database": os.getenv("DUCKDB_DATABASE"),
        },
        max_retries=10,
    )
    conn.execute(query)

```

Explication du code

- **Le décorateur @asset** : Permet à Dagster de reconnaître `taxi_trips` comme un asset.
- **Dépendance `deps=["taxi_trips_file"]`** : Assure que `taxi_trips_file` est matérialisé avant `taxi_trips`.
- **Requête SQL** : Crée ou remplace la table `trips` en important les données du fichier `taxi_trips_file`.
- **Connexion sécurisée avec `backoff`** : Permet d'éviter des conflits d'accès multiples à DuckDB.

Rechargement des définitions

Après avoir ajouté ce nouvel asset, vous devez **recharger les définitions** dans **Dagster UI** :

1. Ouvrez **Dagster UI**.
2. Cliquez sur **"Reload Definitions"**.
3. Vérifiez que `taxi_trips` apparaît dans le graphe des assets avec une flèche indiquant sa dépendance à `taxi_trips_file`.

Matérialisation du pipeline

Dans **Dagster UI**, cliquez sur **"Materialize all"** pour lancer l'exécution des assets :

- **`taxi_trips_file` et `taxi_zones_file` sont exécutés en parallèle.**
- **`taxi_trips` attend que `taxi_trips_file` soit complété** avant d'être matérialisé.

Cela est possible grâce à la déclaration de dépendance `deps=["taxi_trips_file"]`.

Vérification de la matérialisation

Pour confirmer que les données ont bien été chargées dans DuckDB, ouvrez un terminal et exécutez les commandes suivantes :

```
import duckdb
conn = duckdb.connect(database="data/staging/data.duckdb")
conn.execute("SELECT COUNT(*) FROM trips").fetchall()
```

Si tout s'est bien passé, cette commande affichera le nombre de trajets de taxi ingérés.

- ☐ **Félicitations !** Vous avez maintenant construit un pipeline de données complet qui récupère des données depuis une API et les stocke dans une base de données pour analyse.

Pratique : Création d'un asset `taxi_zones`

En utilisant vos connaissances sur la gestion des dépendances entre assets, créez un asset `taxi_zones` qui utilise `taxi_zones_file` pour générer une table `zones` dans DuckDB.

Cet asset doit :

- S'appeler `taxi_zones`.
- Utiliser le fichier `taxi_zones_file` comme source de données.
- Créer une table `zones` dans DuckDB avec les colonnes suivantes :
 - `zone_id`, correspondant à `LocationID`, renommé.
 - `zone`.
 - `borough`.
 - `geometry`, correspondant à `the_geom`, renommé.

Instructions

1. Modifiez le fichier `trips.py` pour ajouter l'asset `taxi_zones`.
2. Assurez-vous que `taxi_zones_file` est correctement référencé comme dépendance.
3. Utilisez une requête SQL pour transformer et charger les données dans DuckDB.
4. Rechargez les définitions dans **Dagster UI**.
5. Matérialisez l'asset `taxi_zones` après `taxi_zones_file`.
6. Vérifiez que la table `zones` a bien été créée dans DuckDB.

- ☐ **Astuce :** Utilisez Dagster UI pour suivre les dépendances et vous assurer que la matérialisation s'exécute correctement.

Assets avec calculs en mémoire

Jusqu'à présent, nous avons orchestré des calculs dans une base de données et effectué des tâches légères en Python, comme le téléchargement de fichiers. Dans cette section, vous allez utiliser Dagster pour orchestrer des calculs en Python et générer un rapport.

Introduction aux assets avec calculs en mémoire

Jusqu'ici, les assets définis impliquaient soit l'exécution de requêtes SQL dans une base de données, soit des opérations légères comme le téléchargement de fichiers. Maintenant, vous allez apprendre à utiliser **Dagster** pour orchestrer des calculs en **Python pur**, afin de transformer vos données et générer des rapports.

Afin de mieux organiser le projet, nous allons **séparer les assets selon leur fonction** : les assets liés à l'analyse seront placés dans un fichier distinct.

1. Créez et ouvrez `metrics.py` dans le répertoire `assets/`.
2. Ajoutez les imports suivants au début du fichier :

```
from dagster import asset
import matplotlib.pyplot as plt
import geopandas as gpd
import duckdb
import os
from . import constants
```

3. Définissez un nouvel asset `manhattan_stats` et ses dépendances :

```
@asset(
    deps=["taxi_trips", "taxi_zones"]
)
def manhattan_stats() -> None:
    """
    Calcule les statistiques des trajets en taxi pour Manhattan et les stocke au format GeoJSON.
    """
    query = """
        SELECT
            zones.zone,
            zones.borough,
            zones.geometry,
            COUNT(1) AS num_trips
        FROM trips
        LEFT JOIN zones ON trips.pickup_zone_id = zones.zone_id
        WHERE borough = 'Manhattan' AND geometry IS NOT NULL
        GROUP BY zone, borough, geometry
    """

    conn = duckdb.connect(os.getenv("DUCKDB_DATABASE"))
    trips_by_zone = conn.execute(query).fetch_df()

    trips_by_zone["geometry"] = gpd.GeoSeries.from_wkt(trips_by_zone["geometry"])
    trips_by_zone = gpd.GeoDataFrame(trips_by_zone)

    with open(constants.MANHATTAN_STATS_FILE_PATH, 'w') as output_file:
        output_file.write(trips_by_zone.to_json())
```

4. Recharger les définitions dans Dagster UI et matérialiser `manhattan_stats`.

5. Vérifiez la création du fichier JSON dans `data/staging/manhattan_stats.geojson`.

Création d'une carte

Créez un asset `manhattan_map` qui dépend de `manhattan_stats`, charge ses données GeoJSON et génère une visualisation.

1. Ajoutez le code suivant à la fin du fichier `metrics.py` :


```
@asset(
    deps=["manhattan_stats"]
)

def manhattan_map() -> None:
    """
    Génère une carte des trajets en taxi à Manhattan et l'enregistre sous forme d'image.
    """

    trips_by_zone = gpd.read_file(constants.MANHATTAN_STATS_FILE_PATH)

    fig, ax = plt.subplots(figsize=(10, 10))
    trips_by_zone.plot(column="num_trips", cmap="plasma", legend=True, ax=ax, edgecolor="black")
    ax.set_title("Nombre de trajets par zone de taxi à Manhattan")

    ax.set_xlim([-74.05, -73.90])
    ax.set_ylim([40.70, 40.82])

    plt.savefig(constants.MANHATTAN_MAP_FILE_PATH, format="png", bbox_inches="tight")
    plt.close(fig)
```

2. Rechargez les définitions dans Dagster UI.
3. Matérialisez `manhattan_map`.
4. Vérifiez la création de l'image `manhattan_map.png` dans `data/outputs/`.

Explication du code

- L'asset `manhattan_map` dépend de `manhattan_stats` et charge ses données GeoJSON.
- Utilise Matplotlib pour générer une visualisation de la répartition des trajets en taxi.
- Stocke l'image générée sous `data/outputs/manhattan_map.png`.

☐ **Félicitations !** Vous avez orchestré un calcul en mémoire et généré une visualisation de données avec Dagster.

Exercice : Création d'un asset `trips_by_week`

Pour mettre en pratique ce que vous avez appris, créez un asset dans `metrics.py` qui :

- S'appelle `trips_by_week`.
- Produit un fichier CSV qui :
 - Est enregistré à l'emplacement défini par `constants.TRIPS_BY_WEEK_FILE_PATH`.
 - Contient les colonnes suivantes :
 - `period` : une chaîne de caractères représentant le dimanche de la semaine agrégée (ex. 2023-03-05).
 - `num_trips` : le nombre total de trajets commencés durant cette semaine.
 - `passenger_count` : le nombre total de passagers sur les trajets de cette semaine.
 - `total_amount` : la somme totale des revenus générés par les trajets de cette semaine.
 - `trip_distance` : la distance totale parcourue en miles pour tous les trajets de cette semaine.

Contraintes supplémentaires (optionnel - challenge avancé)

Si vous souhaitez aller plus loin, imaginez que l'ensemble des données des trajets est trop volumineux pour être chargé entièrement en mémoire, mais qu'une semaine de données peut être traitée confortablement. Réfléchissez à une approche permettant d'adapter l'implémentation en conséquence.

Conseils

- Toutes les notions nécessaires ont déjà été abordées.
- Il existe plusieurs solutions possibles, soit en manipulant la base de données, soit en agrégeant un `DataFrame`.
- Aucun import supplémentaire n'est nécessaire, mais vous pouvez importer tout ce dont vous avez besoin.
- Pour éviter les problèmes liés à la qualité des données, vous pouvez fixer la période d'analyse entre des dates connues (ex. 2023-03-01 à 2023-03-31).
- La fonction `date_trunc` de **DuckDB** permet de tronquer une date à la précision `week`.
- DuckDB permet d'ajouter des intervalles de temps avec `+ INTERVAL '1 week'`.

Exemple de sortie attendue

Le fichier CSV généré pourrait ressembler à ceci :

```
period,num_trips,total_amount,trip_distance,passenger_count
2023-03-05,679681,18495110.72,2358944.42,886486
2023-03-12,686461,19151177.45,2664123.87,905296
2023-03-19,640158,17908993.09,2330611.91,838066
```

❑ **À vous de jouer !** Implémentez cet asset et vérifiez son bon fonctionnement en le matérialisant via **Dagster UI**.

Comprendre l'objet `Definitions`

L'objet `Definitions` regroupe l'ensemble des définitions utilisées par Dagster et les rend accessibles aux outils Dagster. Il permet d'assigner des définitions à une **code location**, chaque code location ne pouvant contenir qu'un seul `Definitions`. Cela permet d'isoler plusieurs projets Dagster sans nécessiter plusieurs déploiements.

Où est défini `Definitions` ?

Dans Dagster, l'objet `Definitions` est défini dans le fichier `__init__.py` à la racine du projet. Ce fichier joue un rôle clé dans la gestion des assets et leur découverte par Dagster.

Ouvrez le fichier `dagster_university/__init__.py` dans votre projet. Il devrait contenir un code similaire à ceci :

```
from dagster import Definitions, load_assets_from_modules

from .assets import trips, metrics

trip_assets = load_assets_from_modules([trips])
metric_assets = load_assets_from_modules([metrics])

defs = Definitions(
    assets=[*trip_assets, *metric_assets]
)
```

Décomposition du fichier `__init__.py`

1. Importation des outils Dagster

```
from dagster import Definitions, load_assets_from_modules
```

Cette ligne introduit l'objet `Definitions` et la méthode `load_assets_from_modules`, utilisée pour charger les assets définis dans le projet.

2. Organisation des assets en modules

```
from .assets import trips, metrics
```

Conformément aux bonnes pratiques recommandées par Dagster, les assets sont stockés dans des modules distincts.

3. Chargement des assets à partir des modules

```
trip_assets = load_assets_from_modules([trips])
metric_assets = load_assets_from_modules([metrics])
```

Cette étape stocke les assets des modules `trips` et `metrics` dans des variables, en les chargeant dynamiquement.

4. Création de l'objet `Definitions`

```
defs = Definitions(
    assets=[*trip_assets, *metric_assets]
)
```

Cette instruction combine les différents assets et les associe à `Definitions`. Cet objet est essentiel, car Dagster recherche automatiquement un `Definitions` dans `__init__.py` lorsqu'on exécute `dagster dev`.

Pourquoi est-ce important ?

- `Definitions` permet à Dagster de charger et de reconnaître les assets du projet.
- Lorsqu'un projet Dagster grandit, il est nécessaire de **mettre à jour `Definitions`** pour y inclure de nouveaux assets, ressources ou capteurs.
- En isolant chaque projet via une **code location**, Dagster permet une meilleure gestion et modularité des assets.

❑ **À retenir** : `Definitions` est le point central permettant à Dagster de comprendre et d'exécuter les assets de votre projet. Toute modification dans la structure des assets doit être répercutée dans cet objet.

Comprendre les code locations dans Dagster

Nous avons parlé des `Definitions`, mais qu'en est-il des **code locations** ? Comment fonctionnent-elles avec les `Definitions` ?

Une **code location** est un regroupement de définitions Dagster, telles que les assets, permettant d'organiser et d'exécuter le code de manière isolée. Une code location est composée de :

- Un **module Python** contenant un objet `Definitions`.
- Un **environnement Python** capable de charger ce module.

Pourquoi les code locations sont-elles utiles ?

Dans l'univers du développement logiciel, un **déploiement** est le processus permettant de rendre une application accessible aux utilisateurs. Un déploiement Dagster inclut tous les éléments nécessaires à son exécution : Python, les packages comme Pandas, l'interface Dagster UI, etc.

À mesure que les organisations grandissent et que plusieurs équipes orchestrent leurs données, un unique déploiement Dagster devient difficile à gérer. Cela peut entraîner plusieurs problèmes :

- Une équipe déployant un changement pourrait provoquer des **interruptions** pour une autre équipe.
- Deux modèles d'apprentissage automatique pourraient nécessiter **des versions différentes d'une même bibliothèque**.
- Une équipe pourrait être contrainte d'utiliser une **ancienne version d'un orchestrateur** en raison de conflits de dépendances.
- Un trop grand nombre d'assets dans un seul environnement peut **rendre la navigation difficile**.

Plutôt que de multiplier les déploiements (ce qui ajouterait de la complexité et des coûts en gestion des infrastructures), Dagster permet de **segmenter le code en plusieurs code locations**. Cela permet aux équipes d'isoler leur code tout en conservant une gestion centralisée des assets.

Exemple : La métaphore de la cuisine

Imaginons que vous ouvriez une **boulangerie** pour produire des cookies à grande échelle. Vous pourriez diviser l'espace en plusieurs cuisines spécialisées :

- Une **cuisine de test** pour expérimenter de nouvelles recettes.
- Un **atelier d'emballage** pour conditionner les cookies.
- Une **zone de décoration** pour ajouter des finitions aux cookies.

Chaque zone fonctionne indépendamment, ce qui évite que des incidents dans l'une n'affectent les autres (ex. un incendie dans la cuisine de test ne mettrait pas en péril la production principale). **C'est exactement ce que permettent les code locations dans Dagster.**

Utilisation des code locations dans Dagster

Chaque **boîte** sur le schéma ci-dessous représente une code location. En séparant les code locations des services principaux de Dagster, le code des utilisateurs est isolé et sécurisé. Cela signifie qu'une code location peut :

- Avoir **sa propre version de Python**.
- Gérer **ses propres dépendances**.
- Exécuter son code sans impacter les autres parties du projet.

Organisation des code locations

Les code locations peuvent être utilisées pour segmenter le code en différentes catégories :

- **Par équipe** (ex. marketing, produit, data science).
- **Par version de Python** (ex. un code legacy en Python 3.9 et un code plus récent en Python 3.11).
- **Par version de dépendances** (ex. un modèle utilisant `PyTorch v1` et un autre `PyTorch v2`).

Même si ces code locations sont **isolées**, elles restent connectées via une même **instance Dagster**. Un asset défini dans une code location peut dépendre d'un asset d'une autre code location.

- **À retenir** : Les code locations permettent d'éviter les conflits entre équipes, de garantir l'évolutivité du projet et d'offrir une meilleure gestion des dépendances sans avoir besoin de multiplier les déploiements Dagster.

Code locations dans Dagster UI

Dans cette section, nous allons explorer comment visualiser et gérer les **code locations** dans l'interface **Dagster UI**.

Visualiser les code locations

1. Accéder aux code locations :

- Dans **Dagster UI**, cliquez sur **Deployments** dans la barre de navigation supérieure.
- Dans l'onglet **Code Locations**, vous verrez la liste des code locations disponibles, avec leur statut, la dernière mise à jour et d'autres informations.

2. Nom des code locations :

- Par défaut, une code location prend le nom du module chargé par Dagster. Dans notre cas, la code location est nommée **dagster_university**, car elle correspond au dossier principal du projet.

Gestion des erreurs de chargement des code locations

Si une erreur survient lors du chargement d'une code location, son statut apparaîtra comme **Failed**.

1. Identifier une erreur :

- Dans l'onglet **Code Locations**, une code location en échec affichera le statut **Failed**.
- Cliquez sur **View Error** pour afficher les logs d'erreur et diagnostiquer le problème.

Rechargement des définitions

Lors de l'ajout ou de la modification de définitions dans votre projet, il peut être nécessaire de **rafraîchir la code location** pour que Dagster prenne en compte les nouvelles versions des fichiers.

Vous pouvez recharger les définitions de deux manières :

1. Depuis l'onglet Deployments :

- Accédez à **Deployments > Code Locations**.
- Cliquez sur le bouton **Reload** à côté de la code location concernée.

2. Depuis la page Global Asset Lineage :

- Accédez à **Global Asset Lineage** dans Dagster UI.
- Cliquez sur **Reload definitions** pour mettre à jour toutes les définitions du projet.

- **À retenir** : Une code location centralise les définitions Dagster et doit être rechargée lorsqu'un asset est ajouté ou modifié afin que Dagster prenne en compte les mises à jour.

Aperçu des Ressources

Précédemment, nous avons exploré les assets, la manière dont ils s'articulent pour former un pipeline de données et comment les exécuter pour les matérialiser.

L'objectif de Dagster est d'offrir une **vue unifiée** sur tous les pipelines de données d'une organisation. Pour y parvenir, Dagster doit être capable d'interagir avec différents services et systèmes utilisés dans ces pipelines, comme le stockage cloud ou les entrepôts de données. Dans cette section, nous allons voir comment accomplir cela en appliquant les meilleures pratiques d'ingénierie logicielle.

Le principe DRY (Don't Repeat Yourself)

Une **bonne pratique** essentielle en développement logiciel est le principe **DRY (Don't Repeat Yourself)**. Ce principe recommande d'éviter la redondance et d'écrire un code qui soit **réutilisable** et **centralisé** plutôt que dupliqué à plusieurs endroits.

En appliquant ce principe à Dagster, on réduit le nombre d'erreurs potentielles, on améliore la lisibilité du code et on facilite l'observabilité des processus et des assets définis.

Tester dans l'environnement de développement

À mesure que les pipelines de données deviennent plus complexes, il devient essentiel de tester les modifications avant leur déploiement en production. Une difficulté en ingénierie des données est que le code et les environnements sont souvent **étroitement couplés**, rendant difficile la validation des changements sans impacter les systèmes en production.

Avec Dagster, il est possible de :

- **Utiliser une base de données locale** au lieu de la base de production.
- **Représenter différemment les connexions externes** selon l'environnement (développement, test, production).
- **Créer une réplique d'un environnement de production** en développement afin de tester les modifications sans risques.

□ **À retenir** : En appliquant ces bonnes pratiques, vous rendez vos pipelines plus robustes, plus lisibles et plus faciles à maintenir.

Comprendre les ressources dans Dagster

Les **resources** sont les outils et services externes utilisés pour créer des assets dans Dagster.

Métaphore des cookies □

Reprenons l'exemple de la fabrication de cookies. Pour réaliser une recette, plusieurs **ustensiles** et **appareils** sont nécessaires :

- Un **bol** et une **cuillère** pour mélanger les ingrédients.
- Un **plateau de cuisson** pour déposer les cookies.
- Un **four** pour les cuire.

Ces éléments sont des **resources**, car ils sont utilisés **à plusieurs reprises** dans le processus. Plutôt que de les dupliquer dans chaque étape, il est plus efficace de les **centraliser et réutiliser**.

Resources dans Dagster

Dans le contexte des pipelines de données, les ressources peuvent inclure :

- Une **API** pour récupérer des données.
- Un **stockage S3** pour sauvegarder des fichiers.
- Une **base de données** comme Snowflake ou BigQuery.
- Un **outil de visualisation** pour afficher les résultats.

Les ressources permettent de **standardiser et centraliser** les connexions avec ces services. L'interface **Dagster UI** facilite également la visualisation et la gestion des ressources utilisées dans le pipeline.

Dans la prochaine section, nous allons refactoriser notre projet pour gérer les connexions DuckDB via une ressource dédiée.

Configuration d'une ressource pour la base de données

Tout au long de ce module, nous avons utilisé **DuckDB** pour stocker et transformer les données. Chaque asset nécessitant une connexion à DuckDB contenait une ligne similaire à celle-ci :

```
conn = backoff(
    fn=duckdb.connect,
    retry_on=(RuntimeError, duckdb.IOException),
    kwargs={
        "database": os.getenv("DUCKDB_DATABASE"),
    },
    max_retries=10,
)
```

Cette approche peut devenir **fragile et source d'erreurs** à mesure que le projet évolue. Une meilleure pratique consiste à **centraliser la gestion de cette connexion** via une **resource Dagster**.

Définition d'une ressource

Lors de la création du projet, un dossier `resources/` contenant un fichier `__init__.py` a été généré. Nous allons y définir une ressource partagée pour gérer la connexion à DuckDB.

Ajoutez le code suivant dans `resources/__init__.py` :

```
from dagster_duckdb import DuckDBResource

database_resource = DuckDBResource(
    database="data/staging/data.duckdb"
)
```

Ce code importe la ressource `DuckDBResource` de la bibliothèque `dagster_duckdb`, puis crée une instance **réutilisable** de cette ressource.

Utilisation des variables d'environnement

Les variables d'environnement sont un moyen standardisé de stocker des configurations sensibles (comme des mots de passe ou des chemins de connexion). Jusqu'ici, nous avons utilisé `os.getenv` pour récupérer ces variables dans le fichier `.env`.

Plutôt que d'inscrire directement le chemin de la base de données, nous allons utiliser **Dagster's EnvVar** pour le rendre plus dynamique. Modifiez `resources/__init__.py` comme suit :

```
from dagster_duckdb import DuckDBResource
from dagster import EnvVar

database_resource = DuckDBResource(
    database=EnvVar("DUCKDB_DATABASE") # Utilisation de la variable d'environnement
)
```

Différence entre EnvVar et os.getenv

- **EnvVar** récupère la valeur à chaque exécution.
- **os.getenv** charge la valeur une seule fois au démarrage.

L'utilisation de **EnvVar** permet de changer la base de données utilisée **sans redémarrer Dagster**.

Mise à jour de Definitions

Les ressources sont des **définitions Dagster** et doivent être ajoutées à l'objet `Definitions` pour être utilisables.

Dans `dagster_university/__init__.py`, ajoutez :

```
from .resources import database_resource
```

Puis, modifiez `Definitions` pour inclure la ressource :

```
defs = Definitions(
    assets=[*trip_assets, *metric_assets],
    resources={
        "database": database_resource,
    },
)
```

Vérification dans Dagster UI

1. **Rendez-vous dans Dagster UI.**
2. Cliquez sur **"Deployment"**, puis **"Code locations"**.
3. **Rechargez les définitions** en cliquant sur **"Reload"**.
4. **Ouvrez la code location**, puis accédez à l'onglet **Resources**.
5. **Vous devriez voir une ressource nommée database listée.**

❑ **À noter** : Pour l'instant, cette ressource n'est pas encore utilisée par les assets. La prochaine section couvrira l'intégration de cette ressource dans les assets existants.

Utilisation des ressources dans les assets

Maintenant que la ressource est définie, nous allons modifier l'asset `taxi_trips` pour l'utiliser.

Avant l'ajout de la ressource

Actuellement, l'asset `taxi_trips` établit une connexion à DuckDB directement :

```
@asset(
    deps=["taxi_trips_file"],
)
def taxi_trips() -> None:
    query = """
        create or replace table taxi_trips as (
            select
                VendorID as vendor_id,
                PULocationID as pickup_zone_id,
                DOLocationID as dropoff_zone_id,
                RatecodeID as rate_code_id,
                payment_type as payment_type,
                tpep_dropoff_datetime as dropoff_datetime,
                tpep_pickup_datetime as pickup_datetime,
                trip_distance as trip_distance,
                passenger_count as passenger_count,
                total_amount as total_amount
            from 'data/raw/taxi_trips_2023-03.parquet'
        );
    """

    conn = backoff(
        fn=duckdb.connect,
        retry_on=(RuntimeError, duckdb.IOException),
        kwargs={
            "database": os.getenv("DUCKDB_DATABASE"),
        },
        max_retries=10,
    )
    conn.execute(query)
```

Après l'ajout de la ressource

Nous allons modifier `taxi_trips` pour utiliser la ressource définie précédemment :

```
@asset(
    deps=["taxi_trips_file"],
)
def taxi_trips(database: DuckDBResource) -> None:
    query = """
        create or replace table taxi_trips as (
            select
                VendorID as vendor_id,
                PULocationID as pickup_zone_id,
                DOLocationID as dropoff_zone_id,
                RatecodeID as rate_code_id,
                payment_type as payment_type,
                tpep_dropoff_datetime as dropoff_datetime,
                tpep_pickup_datetime as pickup_datetime,
                trip_distance as trip_distance,
                passenger_count as passenger_count,
                total_amount as total_amount
            from 'data/raw/taxi_trips_2023-03.parquet'
        );
    """

    with database.get_connection() as conn:
        conn.execute(query)
```

Changements effectués :

1. Importation de DuckDBResource :

```
from dagster_duckdb import DuckDBResource
```

2. Ajout d'un paramètre database: DuckDBResource dans la signature de la fonction.

3. Remplacement de la connexion manuelle à DuckDB par :

```
with database.get_connection() as conn:
    conn.execute(query)
```

☐ Avantages :

- Plus besoin d'utiliser `backoff`, la gestion des connexions est intégrée à `DuckDBResource`.
- Plus de flexibilité pour changer la configuration de la base sans modifier chaque asset.

Avant de continuer

Avant de passer à la suite, assurez-vous de :

- Mettre à jour `assets/trips.py` avec la nouvelle implémentation de `taxi_trips`.
- Recharger les définitions dans Dagster UI.
- Rematéraliser l'asset `taxi_trips`.

☐ Une fois ces étapes terminées, la resource est correctement intégrée et prête à être utilisée dans d'autres assets !

Pratique: Refactorisation des assets pour utiliser les resources

Les assets suivants utilisent la base de données **DuckDB** :

- `taxi_zones`
- `manhattan_stats`
- `trips_by_week`

Mettez à jour ces assets pour qu'ils utilisent la resource **DuckDBResource** au lieu d'une connexion directe à la base de données.

Analyse de l'utilisation des ressources via Dagster UI

Maintenant que vos assets utilisent la ressource `database`, vous pouvez analyser son utilisation dans Dagster UI.

Accéder à l'onglet Resources

1. Ouvrez Dagster UI.
2. Cliquez sur "Deployment", puis sélectionnez la code location `dagster_university`.
3. Accédez à l'onglet "Resources".
4. Vous devriez voir que la colonne **Uses** affiche désormais 4, indiquant que quatre assets utilisent la ressource `database`.
5. Dans la colonne **Name**, cliquez sur `database`.

Affichage des détails d'une ressource

- Cette page contient des informations détaillées sur la ressource, y compris son type et sa configuration.

Visualisation de l'utilisation des ressources

1. Cliquez sur l'onglet "Uses" pour voir quels assets utilisent cette ressource.
2. Cette vue est essentielle pour comprendre quelles ressources sont disponibles et comment elles sont exploitées.

Cas d'utilisation courants

- Identifier les impacts potentiels d'une migration de base de données.
- Analyser l'augmentation des coûts de service et retracer leur origine.

☐ En utilisant cette interface, vous obtenez une vision claire de l'usage des ressources dans vos pipelines Dagster.

Introduction aux Schedules dans Dagster

Jusqu'à présent, vous avez construit un pipeline de données et l'avez matérialisé manuellement. Cependant, le rôle principal d'un orchestrateur est d'exécuter ces processus sans intervention humaine.

Dans cette section, vous allez découvrir la manière la plus simple d'automatiser la matérialisation régulière des assets : **l'utilisation des schedules**.

Comprendre les Schedules

Les **schedules** sont un moyen traditionnel de maintenir vos assets à jour en définissant un **intervalle de temps fixe** pour exécuter un pipeline. Par exemple, un schedule peut être défini pour une exécution **quotidienne, horaire, ou chaque lundi à 9h00**.

Exemple de production automatisée ☐

Si votre boulangerie connaît un grand succès, vous devez cuire vos cookies **tous les jours à 4h00 du matin** pour qu'ils soient frais lorsque les clients arrivent. Un **schedule Dagster** permettrait d'automatiser ce processus sans intervention humaine.

Dans Dagster, un schedule permet d'exprimer **la fréquence d'exécution d'un pipeline**, et chaque exécution (tick) déclenche une matérialisation des assets.

Anatomie d'un Schedule

Un schedule est composé de plusieurs éléments :

- Un **job** qui définit quels assets doivent être matérialisés.
- Une **expression cron** qui définit la fréquence d'exécution.

Nous allons maintenant créer un schedule qui met à jour la plupart des assets **chaque mois**, en accord avec la publication des nouvelles données de taxi par la NYC Taxi & Limo Commission (TLC).

Définition des Jobs

Dans Dagster, **les jobs permettent de sélectionner une partie des assets** à matérialiser. Un **job** est utile lorsqu'on ne veut pas exécuter **tous** les assets à chaque exécution.

Nous allons créer un **job** qui exclut `trips_by_week` car cet asset doit être mis à jour plus fréquemment.

Dans `jobs/__init__.py`, ajoutez :

```
from dagster import AssetSelection, define_asset_job

trips_by_week = AssetSelection.assets("trips_by_week")

trip_update_job = define_asset_job(
    name="trip_update_job",
    selection=AssetSelection.all() - trips_by_week
)
```

Expressions Cron

Les **expressions cron** sont le standard pour définir des fréquences d'exécution dans les orchestrateurs.

Exemple d'expression cron :

```
15 5 * * 1-5
```

→ Cette expression signifie : **Tous les jours ouvrés (lundi-vendredi) à 5h15 du matin.**

Des outils comme **Crontab Guru** permettent de tester des expressions cron avant de les utiliser. Toutefois, il est recommandé de tester directement les schedules dans **Dagster UI** pour s'assurer du bon fonctionnement.

Pratique : Création d'un `weekly_update_job`

Pour mettre en pratique ce que vous avez appris, ajoutez un **job** dans `jobs/__init__.py` qui matérialisera l'asset `trips_by_week`.

Création d'un Schedule

Maintenant que vous connaissez la structure d'un schedule, nous allons en créer un à l'aide de la classe `ScheduleDefinition`.

Ajoutez le code suivant dans `schedules/__init__.py` :

```
from dagster import ScheduleDefinition
from ..jobs import trip_update_job

trip_update_schedule = ScheduleDefinition(
    job=trip_update_job,
    cron_schedule="0 0 5 * *", # Tous les 5 du mois à minuit
)
```

Explication du code :

- **Importation de `ScheduleDefinition`.**
- **Importation du job `trip_update_job`.**
- **Création du schedule `trip_update_schedule`, qui :**
 - Est attaché au job `trip_update_job`.
 - Utilise l'expression cron `0 0 5 * *`, qui exécute le job **chaque 5 du mois à minuit**.

☐ **Votre schedule est maintenant défini !** Passez à la suite pour l'ajouter à Dagster UI.

Pratique : Création d'un `weekly_update_schedule`

Pour mettre en pratique ce que vous avez appris, ajoutez un **schedule** dans `schedules/__init__.py` qui :

- **Se nomme `weekly_update_schedule`.**
- **Matérialise l'asset `trips_by_week`.**
- **S'exécute chaque lundi à minuit.**

Mise à jour de l'objet Definitions

Avant que les jobs et les schedules puissent être utilisés, vous devez les ajouter à l'objet `Definitions`.

Ajout des jobs dans `Definitions`

Revenons aux définitions dans le fichier `__init__.py` à la racine du projet. La première étape consiste à ajouter les jobs pour que les schedules puissent les utiliser.

Ajoutez les imports suivants en haut du fichier :

```
from .jobs import trip_update_job, weekly_update_job
```

Cela permet à Dagster d'importer `trip_update_job` et `weekly_update_job` depuis `jobs/__init__.py`.

Sous la ligne `metric_assets = load_assets_from_modules([metrics])`, ajoutez :

```
all_jobs = [trip_update_job, weekly_update_job]
```

Cela crée une liste `all_jobs` contenant tous les jobs actuels, permettant d'en ajouter d'autres facilement à l'avenir.

Dans l'objet `Definitions`, ajoutez l'argument `jobs` :

```
jobs=all_jobs,
```

À ce stade, le fichier `__init__.py` devrait ressembler à ceci :

```
from dagster import Definitions, load_assets_from_modules

from .assets import trips, metrics
from .resources import database_resource
from .jobs import trip_update_job, weekly_update_job

trip_assets = load_assets_from_modules([trips])
metric_assets = load_assets_from_modules([metrics])

all_jobs = [trip_update_job, weekly_update_job]

defs = Definitions(
    assets=[*trip_assets, *metric_assets],
    resources={
        "database": database_resource,
    },
    jobs=all_jobs,
)
```

Ajout des schedules dans `Definitions`

Maintenant que les jobs sont ajoutés, nous allons ajouter les schedules à `Definitions`.

Ajoutez les imports suivants en haut du fichier :

```
from .schedules import trip_update_schedule, weekly_update_schedule
```

Sous la définition de `all_jobs`, ajoutez :

```
all_schedules = [trip_update_schedule, weekly_update_schedule]
```

Enfin, ajoutez l'argument `schedules` dans `Definitions` :

```
schedules=all_schedules,
```

Le fichier `__init__.py` final devrait ressembler à ceci :

```
from dagster import Definitions, load_assets_from_modules

from .assets import trips, metrics
from .resources import database_resource
from .jobs import trip_update_job, weekly_update_job
from .schedules import trip_update_schedule, weekly_update_schedule

trip_assets = load_assets_from_modules([trips])
metric_assets = load_assets_from_modules([metrics])

all_jobs = [trip_update_job, weekly_update_job]
all_schedules = [trip_update_schedule, weekly_update_schedule]

defs = Definitions(
    assets=[*trip_assets, *metric_assets],
    resources={
        "database": database_resource,
    },
    jobs=all_jobs,
    schedules=all_schedules,
)
```

Visualisation des Jobs et Schedules dans Dagster UI

Comme les autres définitions Dagster, les jobs et schedules peuvent être visualisés et gérés dans **Dagster UI**.

En plus de consulter et gérer ces éléments dans l'interface, l'exécution de `dagster dev` lance également le **dagster-daemon**. Ce processus surveille l'horloge pour exécuter les jobs planifiés et activer les capteurs.

Vérification des Jobs

1. **Rechargez les définitions** dans Dagster UI avant de continuer.
2. Accédez aux jobs en cliquant sur **Overview > Jobs**, ou via **Deployment > Code location > Jobs**.
3. La table affichera les informations suivantes :
 - **Nom** du job
 - **Schedules/Sensors** attachés au job
 - **Dernière exécution**
 - **Historique des exécutions**

En sélectionnant un job, vous pourrez voir son **graphe des assets**.

Vérification des Schedules

1. Accédez aux schedules en cliquant sur **Overview > Schedules**, ou via **Deployment > Code location > Schedules**.
2. La table affichera les détails suivants :
 - **Nom** du schedule
 - **Fréquence d'exécution** (ex. : mensuel, hebdomadaire...)
 - **Statut (activé/désactivé)**
 - **Dernier tick et dernière exécution**

Gestion des Schedules

- **Activation/Désactivation** : utilisez le bouton toggle à côté du nom du schedule.
- **Tester un schedule** : cliquez sur **Test Schedule** et sélectionnez un moment de simulation avant de cliquer sur **Évaluer**.

☐ **Avec ces outils, vous avez maintenant une gestion complète des jobs et schedules dans Dagster UI !**

Introduction aux Partitions dans Dagster

Dans la leçon précédente, vous avez appris à exécuter vos pipelines régulièrement grâce aux **schedules**.

Revenons à notre analogie des cookies : si votre entreprise prospère, vous commencerez à prendre des **commandes en avance**. Préparer chaque cookie dès qu'une commande arrive créerait des problèmes, car :

- **Les commandes du jour sont plus urgentes** que celles prévues pour la semaine prochaine.
- **Le volume de commandes fluctue** : certains jours, vous recevez 100 commandes, et d'autres jours, aucune.

Pour optimiser la production, vous **groupez** les commandes par **jour de retrait**, en ne préparant que celles du jour concerné.

Ce concept de **partitionnement** s'applique également aux **pipelines de données**.

Dans cette leçon, vous apprendrez **pourquoi partitionner vos assets de données** et comment le faire avec Dagster en partitionnant les données des trajets de taxi.

Partitions et Backfills dans Dagster

Qu'est-ce qu'une partition ?

Les partitions permettent de diviser les données en segments plus petits et plus faciles à manipuler. Cette segmentation offre plusieurs avantages :

- **Efficacité des coûts** : Ne traiter que les données nécessaires, stocker les plus récentes dans un stockage rapide et les plus anciennes dans un stockage moins coûteux.
- **Accélération du calcul** : Répartir un grand ensemble de données en parties plus petites améliore les performances des requêtes.
- **Scalabilité** : Distribuer les données sur plusieurs serveurs ou systèmes de stockage, ou exécuter plusieurs partitions en parallèle.
- **Traitement concurrent** : Exécuter plusieurs partitions simultanément pour accélérer le traitement.
- **Débogage rapide** : Tester une partition individuelle avant d'exécuter des analyses sur des plages de données plus larges.

Les partitions sont à la fois un **modèle conceptuel** et une **représentation physique**. Dans Dagster, une seule définition d'asset peut contenir plusieurs partitions.

Par exemple, si nous stockons des commandes de cookies dans une base de données, elles peuvent être toutes dans une même table `orders`. En revanche, si elles sont stockées sur AWS S3 en fichiers parquet, il peut être plus efficace de créer un fichier parquet **par jour**.

En résumé, **les partitions permettent de manipuler des segments spécifiques de données tout en laissant flexibles les méthodes de stockage et d'accès**.

Qu'est-ce qu'un Backfill ?

Un **backfill** est le processus d'exécution des partitions d'assets qui n'existent pas encore ou qui nécessitent une mise à jour.

Les backfills sont couramment utilisés lors de la mise en place d'un pipeline pour la première fois, car les assets doivent être matérialisés pour refléter les données historiques.

Autres cas d'utilisation :

- **Modification de la logique d'un asset** : Si l'algorithme de transformation des données change, un backfill permet de recalculer les valeurs historiques.
- **Rattrapage des partitions manquantes** : Si certaines partitions n'ont pas été générées en raison d'une erreur, un backfill les régénère.

☐ **Dans la section suivante, vous apprendrez à partitionner un asset dans Dagster !**

Création d'une partition

Dans la section précédente, vous avez mis en place des **schedules** mensuels et hebdomadaires pour matérialiser les assets. Vous allez maintenant modifier ces assets pour **partitionner les données des trajets de taxi** et ajouter une nouvelle partition à chaque exécution planifiée.

Heureusement, les données de trajets sont stockées sous forme de **fichiers parquet séparés par mois**, et NYC OpenData fournit des informations historiques remontant à 2009. Cependant, pour optimiser l'utilisation des ressources, nous allons n'ingérer que les données **depuis le début de 2023**.

Définition d'une partition

La première étape consiste à définir une **PartitionDefinition**. Dagster propose des partitions préconstruites pour les données temporelles (**horaires, journalières, hebdomadaires, mensuelles**).

Selon les bonnes pratiques de Dagster, les partitions doivent être définies dans le répertoire `partitions/`. Pour ce projet, elles seront situées dans `partitions/__init__.py`.

Votre projet Dagster contient également un fichier `assets/constants.py`, qui définit les variables `START_DATE` et `END_DATE` permettant de spécifier la période des trajets à ingérer.

Ajoutez le code suivant dans `partitions/__init__.py` :

```

from dagster import MonthlyPartitionsDefinition
from ..assets import constants

start_date = constants.START_DATE
end_date = constants.END_DATE

monthly_partition = MonthlyPartitionsDefinition(
    start_date=start_date,
    end_date=end_date
)

```

❑ Ici, nous créons une partition mensuelle `monthly_partition` en utilisant les dates de début et de fin définies dans `constants.py`.

Nettoyage du stockage existant

Avant de continuer, il est recommandé de **supprimer l'historique de matérialisation** des assets existants. Cela est **nécessaire uniquement en environnement local**.

Exécutez les commandes suivantes dans un terminal :

```
rm $DAGSTER_HOME/storage/taxi_trips_file $DAGSTER_HOME/storage/taxi_trips $DAGSTER_HOME/storage/trips_by_week
```

❑ Vous êtes maintenant prêt à implémenter la partition des assets !

Pratique : Création d'une partition hebdomadaire

Pour mettre en pratique ce que vous avez appris, créez une **partition hebdomadaire** en utilisant `WeeklyPartitionsDefinition` de Dagster avec les mêmes dates de début et de fin.

Ajoutez la définition de cette partition dans `partitions/__init__.py` en respectant la structure existante du projet.

❑ Une fois la partition créée, elle pourra être utilisée pour organiser les données des trajets de taxi par semaine !

Ajout de partitions aux assets

Dans cette section, vous allez mettre à jour les assets dans `assets/trips.py` pour utiliser les partitions.

Commençons par `taxi_trips_file`, dont le code actuel est le suivant :

```

@asset
def taxi_trips_file() -> None:
    """
    Les fichiers parquet bruts du dataset des trajets de taxi. Source : NYC Open Data.
    """
    month_to_fetch = '2023-03'
    raw_trips = requests.get(
        f"https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_{month_to_fetch}.parquet"
    )

    with open(constants.TAXI_TRIPS_TEMPLATE_FILE_PATH.format(month_to_fetch), "wb") as output_file:
        output_file.write(raw_trips.content)

```

Dans cet asset, `month_to_fetch` est défini sur `2023-03` pour récupérer les données des trajets de mars 2023. En utilisant `monthly_partition`, vous pouvez mettre à jour l'asset pour récupérer dynamiquement les données en fonction de la partition.

Ajout de la partition à l'asset :

1. Importer `monthly_partition` depuis le dossier `partitions` en ajoutant la ligne suivante en haut du fichier `assets/trips.py` :

```
from ..partitions import monthly_partition
```

2. Modifier le décorateur @asset pour ajouter le paramètre partitions_def avec monthly_partition :

```
@asset(  
    partitions_def=monthly_partition  
)
```

3. Inclure l'argument context pour accéder aux métadonnées

Dans Dagster, l'argument `context` fournit des métadonnées sur la matérialisation en cours. Ajoutez cet argument à la fonction et activez l'annotation de type en important `AssetExecutionContext` :

```
from dagster import asset, AssetExecutionContext  
  
@asset(  
    partitions_def=monthly_partition  
)  
def taxi_trips_file(context: AssetExecutionContext) -> None:
```

Remarque : L'argument `context` n'est pas spécifique aux partitions. Cependant, c'est la première fois que vous l'utilisez dans Dagster University. Il fournit des informations sur l'exécution de l'asset, telles que la partition en cours de traitement, le job qui a déclenché la matérialisation ou les métadonnées des exécutions précédentes.

4. Utiliser la clé de partition pour récupérer dynamiquement les données du mois correspondant

```
@asset(  
    partitions_def=monthly_partition  
)  
def taxi_trips_file(context: AssetExecutionContext) -> None:  
    partition_date_str = context.partition_key
```

5. Adapter la clé de partition au format attendu par la source de données

Dans NYC OpenData, les fichiers de trajets de taxi sont structurés au format `YYYY-MM`. Cependant, `context.partition_key` fournit la date de la partition sous forme de chaîne `YYYY-MM-DD`. Il faut donc adapter ce format :

```
@asset(  
    partitions_def=monthly_partition  
)  
def taxi_trips_file(context: AssetExecutionContext) -> None:  
    partition_date_str = context.partition_key  
    month_to_fetch = partition_date_str[:3]
```

Après avoir suivi ces étapes, l'asset `taxi_trips_file` doit ressembler au code ci-dessous :

```

from ..partitions import monthly_partition

@asset(
    partitions_def=monthly_partition
)
def taxi_trips_file(context: AssetExecutionContext) -> None:
    """
    Les fichiers parquet bruts du dataset des trajets de taxi. Source : NYC Open Data.
    """

    partition_date_str = context.partition_key
    month_to_fetch = partition_date_str[:~3]

    raw_trips = requests.get(
        f"https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_{month_to_fetch}.parquet"
    )

    with open(constants.TAXI_TRIPS_TEMPLATE_FILE_PATH.format(month_to_fetch), "wb") as output_file:
        output_file.write(raw_trips.content)

```

❑ Désormais, `taxi_trips_file` est partitionné et utilisera la partition correspondante à chaque matérialisation !

Exercice : Partitionner l'asset `taxi_trips`

Pour mettre en pratique ce que vous avez appris, partitionnez l'asset `taxi_trips` par mois en suivant ces directives :

- Étant donné qu'un fichier parquet d'un mois donné peut contenir des données historiques en dehors du mois, il est recommandé de partitionner **par le mois du fichier parquet** et non par le mois du trajet.
- À chaque partition, insérez les nouvelles données dans la table `taxi_trips`.
- Ajoutez une colonne `partition_date` pour représenter de quelle partition provient chaque enregistrement.

Suppression de la table existante

Avant d'appliquer ces modifications, vous devez supprimer l'ancienne table `taxi_trips` en raison de l'ajout de la nouvelle colonne `partition_date`. Pour cela, exécutez la commande suivante dans un REPL Python ou un script temporaire :

```

import duckdb
conn = duckdb.connect(database="data/staging/data.duckdb")
conn.execute("drop table trips;")

```

Adaptation de la requête SQL

Étant donné que la table `taxi_trips` existera après la première matérialisation d'une partition, la requête SQL devra être mise à jour pour :

1. Créer la table `taxi_trips` si elle n'existe pas déjà.
2. Supprimer les anciennes données correspondant à `partition_date` pour éviter les doublons lors d'un backfill.
3. Insérer les nouveaux enregistrements du fichier parquet correspondant au mois de la partition.

❑ Modifiez `taxi_trips` pour appliquer cette partition et validez votre implémentation dans Dagster UI !

Création d'un job planifié avec partitions

Dans la section précédente, vous avez créé le job `trip_update_job` qui met à jour la plupart de vos assets. Ce job était programmé pour s'exécuter le cinquième jour de chaque mois à minuit.

Maintenant que les assets pertinents sont partitionnés, la planification peut être modifiée afin de ne récupérer que les nouvelles données du dernier mois au lieu de rafraîchir l'intégralité des assets. C'est une **meilleure pratique** qui optimise les ressources et réduit le temps de calcul.

Actuellement, `trip_update_job` dans `jobs/__init__.py` doit ressembler à ceci :


```
trip_update_job = define_asset_job(  
    name="trip_update_job",  
    selection=AssetSelection.all() - AssetSelection.assets(["trips_by_week"]),  
)
```

Ajout de la partition au job

1. Importer la partition mensuelle depuis le fichier partitions :

```
from ..partitions import monthly_partition
```

2. Ajouter le paramètre `partitions_def` dans le job pour qu'il utilise `monthly_partition` :

```
partitions_def=monthly_partition,
```

Après modifications, le job doit ressembler à ceci :

```
from dagster import define_asset_job, AssetSelection  
from ..partitions import monthly_partition  
  
trips_by_week = AssetSelection.assets("trips_by_week")  
  
trip_update_job = define_asset_job(  
    name="trip_update_job",  
    partitions_def=monthly_partition, # partitions ajoutées ici  
    selection=AssetSelection.all() - trips_by_week  
)
```

Pratique : Partitionner l'asset `trips_by_week`

Pour mettre en pratique ce que vous avez appris, mettez à jour `weekly_update_job` et `trips_by_week` pour qu'ils soient partitionnés de manière hebdomadaire.

Utilisez votre définition `weekly_partition` existante provenant de l'exercice précédent.

Instructions :

- Ajoutez `partitions_def=weekly_partition` à l'asset `trips_by_week`.
 - Assurez-vous que `weekly_update_job` utilise également `weekly_partition`.
 - Vérifiez que la clé de partition est correctement intégrée dans la logique de récupération et d'insertion des données.
 - Testez la matérialisation de partitions distinctes dans Dagster UI.
- ☐ Une fois terminé, vérifiez dans l'interface de Dagster que les partitions s'affichent correctement et que le traitement fonctionne comme prévu.

Partitions et Backfills dans l'interface Dagster UI

Maintenant que vous avez créé vos partitions, utilisez ces partitions dans les assets et mis à jour les jobs avec des partitions pour créer un schedule partitionné, examinons l'affichage dans l'interface Dagster UI. Si vous avez toujours `dagster dev` en cours d'exécution, vous devrez **recharger les définitions** pour assurer la visibilité des partitions.

Visualisation et matérialisation des assets avec partitions

Étape 1

Naviguez vers **Assets > Asset lineage**. Comme vous le verrez dans le graphe des assets, les assets `taxi_trips` contiennent maintenant des informations de partition.

Étape 2

En examinant de plus près l'asset `taxi_trips_file`, vous verrez trois partitions représentant les trois mois inclus dans la partition :

- 0 indique que **zéro partitions ont été matérialisées avec succès**.

- `O All` indique que **toutes les partitions (trois) ne sont pas encore matérialisées**.
- `△ 0` représente qu'**aucune partition n'a échoué**.

Cette information est utile pour obtenir un aperçu rapide de l'état de votre asset.

Étape 3

En cliquant sur **Materialize all**, une fenêtre contextuelle s'affichera avec les informations de la partition.

Cela vous permet de choisir **quelles partitions matérialiser** ou de **lancer un backfill** pour matérialiser plusieurs partitions en une seule fois.

Lancement d'un Backfill

Étape 1

Pour démarrer le backfill, cliquez sur **Launch backfill** dans la fenêtre contextuelle précédente. Par défaut, l'intervalle de dates couvrira toute la plage définie.

Ensuite, allez dans **Overview > Backfills** pour voir les informations sur le backfill.

Étape 2

Cliquez sur un backfill pour ouvrir sa page de détails. Cette page affiche chaque asset en cours de matérialisation, ainsi que son état.

Étape 3

Retournez sur la page **Global Asset Lineage (Assets > Asset lineage)**.

Dans le graphe des assets, vous verrez que le statut des assets partitionnés et non partitionnés a été mis à jour.

Visualisation des détails des partitions d'un asset

Étape 1

Dans le graphe des assets, cliquez sur l'asset `taxi_trips` et ouvrez-le dans l'**Asset Catalog**. Chaque mois sera listé comme une partition examinable.

Étape 2

Sélectionnez `2023-03-01` pour voir une partition spécifique. Vous pourrez alors consulter **les informations spécifiques à cette partition**, y compris son lien avec les données sources (`taxi_trips_file`).

Introduction aux capteurs dans Dagster

Précédemment, vous avez écrit un schedule pour exécuter vos pipelines tous les mois. L'utilisation de schedules n'est qu'une des nombreuses manières de déclencher la matérialisation des assets dans Dagster.

Les exécutions basées sur des événements sont courantes dans la réalité. Prenons l'exemple de notre boulangerie : nous recevons des livraisons d'ingrédients et devons les déplacer en cuisine. De la même manière, les pipelines de données doivent parfois réagir à des événements externes.

Dans Dagster, vous pouvez utiliser **les capteurs (sensors)** pour automatiser vos pipelines et les rendre réactifs à ces événements.

Qu'est-ce qu'un capteur ?

Les capteurs (**sensors**) sont un moyen de surveiller un événement spécifique et de déclencher des exécutions en fonction de celui-ci. Ils fonctionnent en exécutant périodiquement une logique pour déterminer si une exécution doit être lancée. Par défaut, les capteurs dans Dagster interrogent toutes les **30 secondes**.

Les capteurs sont particulièrement utiles pour **déclencher la matérialisation d'un asset après qu'un événement se soit produit**, par exemple :

- L'arrivée d'un nouveau fichier dans un emplacement spécifique (ex: Amazon S3).
- La matérialisation d'un autre asset.
- La libération d'un slot de travail dans un système externe.

Contexte de cette section

Dans cette section, nous supposons un scénario hypothétique où les parties prenantes de vos rapports sur les trajets de taxi effectuent régulièrement des analyses exploratoires sur le nombre de trajets dans certains quartiers (ex: Manhattan ou Brooklyn) à des plages horaires spécifiques.

Elles cherchent à répondre à des questions comme :

« Comment les fêtes de fin d'année affectent-elles le trafic aux heures de pointe à Manhattan ? »

Ce type de demandes répétitives mobilise une partie importante de votre temps. Pour automatiser ces analyses et permettre un **reporting en libre-service**, vous décidez de mettre en place un **capteur Dagster**.

Vous avez conçu un formulaire d'intake qui génère un fichier **JSON structuré** décrivant la requête et l'insère dans le dossier `data/requests` de votre projet Dagster.

Objectifs de l'automatisation avec un capteur

Nous allons maintenant configurer un capteur qui surveille l'apparition de nouvelles demandes et exécute automatiquement les analyses associées.

Lorsqu'un **nouveau fichier JSON** représentant une demande d'analyse arrive dans le répertoire, **le capteur est déclenché et matérialise l'asset** correspondant. Si toutes les demandes ont déjà été traitées, **aucune nouvelle matérialisation ne sera effectuée**.

Pour répondre à ces besoins, nous allons :

1. **Écrire une logique permettant de personnaliser la matérialisation des assets en fonction des requêtes.**
2. **Créer un nouvel asset qui génère un rapport personnalisé par requête.**
3. **Ajouter un capteur pour surveiller les nouvelles requêtes et déclencher l'analyse automatiquement.**

Configuration de la création d'assets

La configuration de la création d'assets n'est pas spécifique aux capteurs, mais lorsqu'on matérialise des assets, il peut être nécessaire de personnaliser certains aspects de la logique en fonction du contexte d'exécution. Par exemple, personnaliser un email en fonction du destinataire ou adapter une analyse en fonction d'une plage de dates.

Dans notre cas, nous voulons personnaliser la matérialisation d'un asset en fonction du **quartier et de la plage de dates** spécifiés par l'utilisateur dans sa requête.

Ces configurations sont généralement ajoutées **au niveau de l'exécution** et peuvent être utilisées dans les plannings et capteurs. De plus, elles peuvent être configurées manuellement via l'interface utilisateur de Dagster lors du lancement d'une exécution.

Implémentation de la configuration personnalisée

1. **Créer un fichier `requests.py`** dans le dossier `assets`.

2. **Ajouter l'import suivant** en haut du fichier :

```
from dagster import Config
```

Cet import permet d'utiliser `Config` comme classe de base pour la configuration personnalisée.

3. **Définir une nouvelle classe `AdhocRequestConfig`** qui contiendra les informations suivantes :

- `filename` : Nom du fichier JSON contenant la requête.
- `borough` : Le quartier de New York à analyser (**Manhattan, Brooklyn, Queens, Bronx, Staten Island**).
- `start_date` : Début de la plage de dates demandée (**format YYYY-MM-DD**).
- `end_date` : Fin de la plage de dates demandée (**format YYYY-MM-DD**).

Le fichier `requests.py` final devrait ressembler à ceci :

```
from dagster import Config

class AdhocRequestConfig(Config):
    filename: str
    borough: str
    start_date: str
    end_date: str
```

- Cette classe nous permettra de passer des configurations dynamiques aux matérialisations d'assets en fonction des demandes spécifiques des utilisateurs.

Création d'un asset déclenché par un capteur

Maintenant que vous avez défini comment l'asset peut être matérialisé, créons l'asset de rapport à la demande.

1. Ajoutez les imports suivants à `requests.py` :

```
from dagster import asset, Config
from dagster_duckdb import DuckDBResource

import matplotlib.pyplot as plt

from . import constants
```

2. Créez un nouvel asset nommé `adhoc_request` avec les arguments suivants :

- `config`, annoté avec `AdhocRequestConfig`.
- Dépendances sur les assets `taxi_zones` et `taxi_trips`.
- `database`, annoté avec `DuckDBResource` pour pouvoir interroger DuckDB.

```
@asset(
    deps=["taxi_zones", "taxi_trips"]
)
def adhoc_request(config: AdhocRequestConfig, database: DuckDBResource) -> None:
```

3. Générez le nom du fichier de sortie en utilisant un template fourni dans `assets/constants.py`, en supprimant l'extension `.json` :

```
file_path = constants.REQUEST_DESTINATION_TEMPLATE_FILE_PATH.format(config.filename.split('.')[0])
```

4. Écrivez une requête SQL qui :

- Filtre les trajets qui n'ont pas commencé dans le borough spécifié.
- Agrège les données par jour de la semaine et heure du jour.

```

query = f"""
select
    date_part('hour', pickup_datetime) as hour_of_day,
    date_part('dayofweek', pickup_datetime) as day_of_week_num,
    case date_part('dayofweek', pickup_datetime)
        when 0 then 'Sunday'
        when 1 then 'Monday'
        when 2 then 'Tuesday'
        when 3 then 'Wednesday'
        when 4 then 'Thursday'
        when 5 then 'Friday'
        when 6 then 'Saturday'
    end as day_of_week,
    count(*) as num_trips
from trips
left join zones on trips.pickup_zone_id = zones.zone_id
where pickup_datetime >= '{config.start_date}'
and pickup_datetime < '{config.end_date}'
and pickup_zone_id in (
    select zone_id
    from zones
    where borough = '{config.borough}'
)
group by 1, 2
order by 1, 2 asc
"""

```

5. Exécutez la requête dans DuckDB et stockez les résultats dans un DataFrame :

```

with database.get_connection() as conn:
    results = conn.execute(query).fetch_df()

```

6. Générez une visualisation avec Matplotlib pour afficher le nombre de trajets en fonction de l'heure et du jour de la semaine :

```

fig, ax = plt.subplots(figsize=(10, 6))

results_pivot = results.pivot(index="hour_of_day", columns="day_of_week", values="num_trips")
results_pivot.plot(kind="bar", stacked=True, ax=ax, colormap="viridis")

ax.set_title(f"Number of trips by hour of day in {config.borough}, from {config.start_date} to {config.end_date}")
ax.set_xlabel("Hour of Day")
ax.set_ylabel("Number of Trips")
ax.legend(title="Day of Week")

plt.xticks(rotation=45)
plt.tight_layout()

```

7. Enregistrez le graphique sous forme de fichier image en utilisant file_path :

```

plt.savefig(file_path)
plt.close(fig)

```

8. Vérifiez que votre fichier `requests.py` ressemble à ceci :

```

from dagster import Config, asset
from dagster_duckdb import DuckDBResource

import matplotlib.pyplot as plt

from . import constants

class AdhocRequestConfig(Config):
    filename: str
    borough: str
    start_date: str
    end_date: str

@asset(
    deps=["taxi_zones", "taxi_trips"]
)
def adhoc_request(config: AdhocRequestConfig, database: DuckDBResource) -> None:
    """
    The response to an request made in the `requests` directory.
    See `requests/README.md` for more information.
    """

    file_path = constants.REQUEST_DESTINATION_TEMPLATE_FILE_PATH.format(config.filename.split('.')[0])

    query = f"""
    select
        date_part('hour', pickup_datetime) as hour_of_day,
        date_part('dayofweek', pickup_datetime) as day_of_week_num,
        case date_part('dayofweek', pickup_datetime)
            when 0 then 'Sunday'
            when 1 then 'Monday'
            when 2 then 'Tuesday'
            when 3 then 'Wednesday'
            when 4 then 'Thursday'
            when 5 then 'Friday'
            when 6 then 'Saturday'
        end as day_of_week,
        count(*) as num_trips
    from trips
    left join zones on trips.pickup_zone_id = zones.zone_id
    where pickup_datetime >= '{config.start_date}'
    and pickup_datetime < '{config.end_date}'
    and pickup_zone_id in (
        select zone_id
        from zones
        where borough = '{config.borough}'
    )
    group by 1, 2
    order by 1, 2 asc
    """

    with database.get_connection() as conn:
        results = conn.execute(query).fetch_df()

    fig, ax = plt.subplots(figsize=(10, 6))

    results_pivot = results.pivot(index="hour_of_day", columns="day_of_week", values="num_trips")
    results_pivot.plot(kind="bar", stacked=True, ax=ax, colormap="viridis")

    ax.set_title(f"Number of trips by hour of day in {config.borough}, from {config.start_date} to {config.end_date}")
    ax.set_xlabel("Hour of Day")

```

```
ax.set_ylabel("Number of Trips")
ax.legend(title="Day of Week")

plt.xticks(rotation=45)
plt.tight_layout()

plt.savefig(file_path)
plt.close(fig)
```

Création d'un job

Ensuite, nous allons créer un job qui matérialise le nouvel asset `adhoc_request`. Ce job sera presque identique à ceux de la **Leçon 7**.

1. Ajoutez les lignes suivantes dans `jobs/__init__.py` pour créer un job dédié aux requêtes ad hoc :

```
from dagster import define_asset_job, AssetSelection

adhoc_request = AssetSelection.assets(["adhoc_request"])

adhoc_request_job = define_asset_job(
    name="adhoc_request_job",
    selection=adhoc_request,
)
```

2. Mettez à jour le job existant `trip_update_job`.

Actuellement, `trip_update_job` sélectionne **tous les assets** avec `AssetSelection.all()`, ce qui inclurait `adhoc_request`. Cependant, nous ne voulons pas que ce job traite les requêtes ad hoc.

Comme nous avons déjà exclu `trips_by_week`, nous allons également exclure `adhoc_request` dans la sélection d'assets.

Modifiez `trip_update_job` comme suit :

```
trip_update_job = define_asset_job(
    name="trip_update_job",
    partitions_def=monthly_partition,
    selection=AssetSelection.all() - trips_by_week - adhoc_request
)
```

□ Ces modifications permettent de gérer séparément les requêtes ad hoc et les mises à jour régulières du pipeline.

Création d'un capteur (sensor)

Avec le nouvel asset et la configuration en place, créons un capteur qui réagit aux événements en matérialisant l'asset `adhoc_request`.

Ce capteur va surveiller les nouveaux fichiers JSON déposés dans le répertoire `data/requests`, puis déclencher une exécution du job `adhoc_request_job` avec les paramètres extraits du fichier JSON correspondant.

Cursors des capteurs

Un **curseur** est une valeur stockée utilisée pour gérer l'état du capteur. Le capteur que nous allons construire utilise un curseur pour garder une trace des requêtes déjà traitées.

Le capteur suivra les étapes suivantes :

1. Lire son curseur (l'état enregistré précédemment).
2. Vérifier si de nouveaux fichiers JSON sont présents dans `data/requests`.
3. Comparer les fichiers actuels à ceux déjà traités.
4. Si de nouveaux fichiers sont détectés, déclencher une exécution pour chaque fichier.
5. Mettre à jour le curseur avec les fichiers déjà traités.

Implémentation du capteur

1. Ajoutez les imports suivants dans `sensors/__init__.py`:

```
from dagster import (
    RunRequest,
    SensorEvaluationContext,
    SensorResult,
    sensor,
)

import os
import json

from ..jobs import adhoc_request_job
```

2. Définissez le capteur `adhoc_request_sensor`:

```
@sensor(
    job=adhoc_request_job
)
def adhoc_request_sensor(context: SensorEvaluationContext):
    PATH_TO_REQUESTS = os.path.join(os.path.dirname(__file__), "../..", "data/requests")

    previous_state = json.loads(context.cursor) if context.cursor else {}
    current_state = {}
    runs_to_request = []

    for filename in os.listdir(PATH_TO_REQUESTS):
        file_path = os.path.join(PATH_TO_REQUESTS, filename)
        if filename.endswith(".json") and os.path.isfile(file_path):
            last_modified = os.path.getmtime(file_path)

            current_state[filename] = last_modified

            # Vérifie si le fichier est nouveau ou a été modifié
            if filename not in previous_state or previous_state[filename] != last_modified:
                with open(file_path, "r") as f:
                    request_config = json.load(f)

                runs_to_request.append(RunRequest(
                    run_key=f"adhoc_request_{filename}_{last_modified}",
                    run_config={
                        "ops": {
                            "adhoc_request": {
                                "config": {
                                    "filename": filename,
                                    **request_config
                                }
                            }
                        }
                    }
                ))

    return SensorResult(
        run_requests=runs_to_request,
        cursor=json.dumps(current_state)
    )
```

□ Ce capteur surveille le dossier `data/requests`, déclenche une exécution pour chaque nouveau fichier JSON et met à jour son état à chaque itération.

Mise à jour de l'objet Definitions

Comme pour les autres définitions de Dagster, la dernière étape consiste à ajouter le capteur, son asset associé et son job dans l'objet `Definitions`. Cela se fera dans le fichier `__init__.py` de niveau supérieur.

1. Ajoutez les nouvelles importations dans `__init__.py`:

```
from dagster import Definitions, load_assets_from_modules

from .assets import trips, metrics, requests
from .resources import database_resource
from .jobs import trip_update_job, weekly_update_job, adhoc_request_job
from .schedules import trip_update_schedule, weekly_update_schedule
from .sensors import adhoc_request_sensor
```

2. Ajoutez l'asset `request_assets` en chargeant les assets depuis `requests`:

```
request_assets = load_assets_from_modules([requests])
```

3. Ajoutez `adhoc_request_job` à `all_jobs`:

```
all_jobs = [trip_update_job, weekly_update_job, adhoc_request_job]
```

4. Ajoutez `adhoc_request_sensor` à `all_sensors`:

```
all_sensors = [adhoc_request_sensor]
```

5. Mettez à jour l'objet `Definitions`:

```
defs = Definitions(
    assets=[*trip_assets, *metric_assets, *request_assets],
    resources={
        "database": database_resource,
    },
    jobs=all_jobs,
    schedules=all_schedules,
    sensors=all_sensors
)
```

❑ Votre sensor `adhoc_request_sensor` est maintenant enregistré et prêt à être utilisé dans Dagster !

Sensors dans l'interface Dagster

Maintenant que le capteur est construit, explorons son affichage dans l'interface utilisateur de Dagster.

Étape 1 : Vérification de l'asset `adhoc_request`

- Accédez à la page **Global Asset Lineage**.
- Vous devriez voir le nouvel asset `adhoc_request`.
- **Remarque** : Si l'asset n'apparaît pas, cliquez d'abord sur **Reload definitions**.

Étape 2 : Vérification du capteur

- Accédez à **Overview > Sensors**.
- Dans cet onglet, vous verrez `adhoc_request_sensor`.
- **Remarque** : Par défaut, tous les capteurs et plannings sont désactivés lorsqu'ils sont chargés pour la première fois. Vous verrez donc que le bouton **Running** est désactivé.

Étape 3 : Consultation des détails du capteur

- Cliquez sur `adhoc_request_sensor` pour ouvrir sa page de détails.
- Cette page fournit des informations détaillées sur le capteur, notamment :
 - Les jobs qui l'utilisent
 - L'historique des exécutions (ticks et runs)
 - D'autres paramètres liés à son fonctionnement

☐ **Votre capteur est maintenant visible dans Dagster !** Nous verrons ensuite comment l'activer et l'exécuter automatiquement.

Activation du capteur

Les capteurs sont désactivés par défaut, vous devez donc activer le capteur avant de pouvoir l'utiliser. Cette section vous guidera pour activer le capteur dans l'interface Dagster.

Étape 1 : Création d'un fichier de requête

- Accédez au dossier `data/requests`.
- Vous y trouverez un fichier `README` contenant des instructions pour déclencher le capteur.
- Utilisez l'exemple `.json` du `README` pour créer un fichier nommé `january-staten-island.json` contenant le code suivant :

```
{
  "start_date": "2023-01-10",
  "end_date": "2023-01-25",
  "borough": "Staten Island"
}
```

Étape 2 : Tester le capteur dans l'interface Dagster

Dagster permet de tester les capteurs directement depuis l'interface, ce qui est utile pour simuler des capteurs qui sont déclenchés trop fréquemment ou trop rarement.

- Si vous avez quitté la page des détails du capteur `adhoc_request_sensor`, retournez-y.
- Cliquez sur le bouton **Test Sensor** en haut à droite de la page. Cela permet de produire un tick unique sans activer le capteur.
- Cliquez sur **Evaluate** pour commencer le test.

Étape 3 : Examiner les résultats du test

- Une fois le test terminé, les résultats s'afficheront.
- Cliquez sur **Open in Launchpad** pour naviguer vers le job `adhoc_request_job`.

Étape 4 : Lancer le job

- Dans **Launchpad**, observez que la configuration a été pré-remplie avec les données du fichier `january-staten-island.json`.
- Cliquez sur **Launch Run** en bas de la page pour soumettre l'exécution du job.

Étape 5 : Vérification des résultats

- Une fois l'exécution terminée, accédez au dossier `data/outputs` dans le répertoire du projet.
- Vous devriez voir un graphique à barres représentant le nombre de trajets par heure dans **Staten Island** en janvier 2023.

☐ **Votre capteur est maintenant activé et fonctionne comme prévu !**